Quantum Mechanics with QuTiP

Andrew M.C. Dawes

June 24, 2026

Contents

0.1 Chapter 1

An introduction to the Jupyter Notebook and some practice with probability ideas from Chapter 1.

0.1.1 1.1 Probability

1.1.1 Moments of Measured Data

The Jupyter Notebook has two primary types of cells "Markdown" cells for text (like this one) and "Code" cells for running python code. The cell below this one is a code cell that loads the plotting functions into the `plt` namespace and loads several functions from the `numpy` library. The last line requests that all plots show up inline in the notebook (instead of in other windows or as files on your computer).

```
import matplotlib.pyplot as plt
from numpy import array, sin, sqrt, dot, outer
%matplotlib inline
```

Arrays of data, and their average and sum in Python. We use some definitions from `numpy`. Notice the way these operators can be applied to arrays (with the "." operator).

```
x = array([1,2,3])
```

```
x.sum()
```

```
x.mean()
```

The formal definition (and to make sure we match with the book) is to take the sum and divide by the number of items in the sample:

```
x.sum()/len(x)
```

Higher order moments Operate on the sample, python does this element-by-element, then do the same thing as above. You may be surprised that to raise a power is "**" instead of "^". This is a difference between python and other languages, just something to keep in mind!

```
x**2
```

```
(x**2).sum() # Note the parenthesis!
```

```
(x**2).sum()/len(x)
```

```
(x**2).mean()
```

It works for functions too

`sin(x)` will find the `sin` of each element in the array `x`:

```
sin(x)
```

```
sin(x).sum()/len(x)
```

```
sin(x).mean()
```

Variance

```
x.var()  # Variance

x.std()  # Standard Deviation
```

We can test if two quanties are equal with the == operator. Not the same as = since the = is an assignment operator. This will trip you up if you are new to programming, but you'll get over it.

```
x.std()**2 == x.var()  # Related by a square root
```

Example 1.1

```
x_m = array([9,5,25,23,10,22,8,8,21,20])

x_m.mean()

(x_m**2).sum()/len(x_m)

sqrt(281.3 - (15.1)**2)

x_m.std()

sqrt(281.3 - (15.1)**2) == x_m.std()
```

Close enough!

1.1.2 Probability

Example 1.2

This is an illustration of how to implement the histogram from Example 1.2 in the text. Note the use of setting the number of bins. The `hist` command will pick for you, and you should try other values to see the impact. There is no one correct value, but the too many bins doesn't illustrate clusters of data, and too-few bins tends to oversimplify the data.

```
n, bins, patches = plt.hist(x_m,bins=7)
```

The hist function has several possible arguments, we use bins=7 to match the example.

```
# an array of the counts in each bin:
n

n/10.0*array([6,9,12,15,18,21,24])  # counts times each bin -center value

# sum of the last cell should be the mean:
sum(_)

n/10.0*array([6,9,12,15,18,21,24])**2  # counts times each bin -center value

# sum of the last cell should be the second moment:
sum(_)
```

Both of these results are close to the previous value, but not exact. Remember, the historgram is a representation of the data and the agreement will improve for larger data sets.

0.1.2 1.2 Linear Algebra

1.2.1 Vectors and Basis sets

We'll use the qutip library later, even though this is all just linear algebra. For now, try using standard-python for vector math:

```
rvec = array([1,2])  # A row vector

rvec

cvec = array([[1],[2]])  # A column vector

cvec

cvec*rvec # Actually the outer product:

rvec*cvec # still the outer product... so this simple ‘*‘ doesn’t respect the rules of linear
```

The `dot` function properly computes the dot product that we know and love from workshop physics:

```
dot(rvec,cvec)

outer(cvec,rvec)

# This doesn’t work if you try to run it because ‘dot‘ knows what shape the vectors should be

# dot(cvec,rvec)
```

This probably isn't your first error message, but it's important to look at what Python is telling you. First, it lists the type of error (ValueError). Then it shows where the error occured (in the dot function). This is helpful when you have larger cells with more lines in them. Obviously we already know what line caused this error since there is only one. Finally, the error is explained as follows: the shapes are not aligned, meaning the vectors don't have the right dimensions for a dot product. Some error messages are more helpful than others, but they all look like this. An error and a traceback.

0.2 2.3 Find the eigenvalues and vectors of $\mathbf{J}_\theta$ for θ=45

```
from qutip import *
from numpy import cos, sin, pi, deg2rad

qm = Qobj([[cos(pi/4)**2,cos(pi/4)*sin(pi/4)],[cos(pi/4)*sin(pi/4),sin(pi/4)**2]])

qm

qm.eigenstates()

from numpy import sqrt, exp
1/sqrt(2)

phi = 1.5
Jphi = Qobj([[exp(1j)*phi,0],[0,1]])
Jphi

hvec = Qobj([[1],[0]])
hvec

vvec = Qobj([[0],[1]])
vvec

Jphi*hvec

Jphi*vvec
```

0.3 Chapter 4 in-class problems

0.3.1 Using what you learned in Lab, answer questions 4.7, 4.8, 4.10, 4.11, 4.12, and 4.13

```
from numpy import sqrt, pi, cos, sin
from qutip import *
```

Remember, these states are represented in the HV basis:

```
H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
P45 = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
M45 = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
R = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
L = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])
```

The sim_transform function creates the matrix $\bar{\mathbf{S}}$ that can convert from one basis to another. As an example, it will create the tranform matrix to convert from HV to ± 45 if you run:

```
Shv45 = sim_transform(H,V,P45,M45)    #  creates the matrix Shv45
```

Then you can convert a ket from HV to ± 45 by applying the Shv45 matrix:

```
Shv45*H    #  will convert H from the HV basis to the \pm45 basis
```

To convert operators, you have to sandwich the operator between $\bar{\mathbf{S}}$ and $\bar{\mathbf{S}}^{\dagger}$:

```
Shv45*Ph*Shv45.dag()     #  converts Ph from HV basis to the \pm45 basis.
```

```
def sim_transform(o_basis1, o_basis2, n_basis1, n_basis2):
    a = n_basis1.dag()*o_basis1
    b = n_basis1.dag()*o_basis2
    c = n_basis2.dag()*o_basis1
    d = n_basis2.dag()*o_basis2
    return Qobj([[a,b],[c,d]])
```

```
ShvLR = sim_transform(H,V,L,R)
```

```
ShvLR*H
```

```
ShvLR*V
```

0.3.2 4.11: Express $\hat{R}_p(\theta)$ in ± 45 basis

```
def Rp(theta):
    return Qobj([[cos(theta), -sin(theta)],[sin(theta),cos(theta)]]).tidyup()
```

```
Rp(1.3)
```

```
Shv45 = sim_transform(H,V,P45,M45)
```

0.3.3 4.12:

```
Rp45 = Shv45*Rp(pi/4)*Shv45.dag()
```

```
Rp45*Shv45*P45 == Shv45*V   # convert P45 to the \pm45 basis
```

```
Rp45* Qobj([[1],[0]])
```

```
ShvLR = sim_transform(H,V,L,R)
```

```
ShvLR*Rp(pi/4)*ShvLR.dag()
```

0.4 Chapter 4 in-class problems - Solutions

0.4.1 Using what you learned in Lab, answer questions 4.7, 4.8, 4.10, 4.11, 4.12, and 4.13

```
from numpy import sin,cos,sqrt,pi
from qutip import *
```

Remember, these states are represented in the HV basis:

```
H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
P45 = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
M45 = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
R = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
L = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])
```

The sim_transform function creates the matrix $\bar{\mathbf{S}}$ that can convert from one basis to another. As an example, it will create the tranform matrix to convert from HV to ± 45 if you run:

```
Shv45 = sim_transform(H,V,P45,M45)    #  creates the matrix Shv45
```

Then you can convert a ket from HV to ± 45 by applying the Shv45 matrix:

```
Shv45*H    #  will convert H from the HV basis to the \pm45 basis
```

To convert operators, you have to sandwich the operator between $\bar{\mathbf{S}}$ and $\bar{\mathbf{S}}^{\dagger}$:

```
Shv45*Ph*Shv45.dag()     #  converts Ph from HV basis to the \pm45 basis.
```

```
def sim_transform(o_basis1, o_basis2, n_basis1, n_basis2):
    a = n_basis1.dag()*o_basis1
    b = n_basis1.dag()*o_basis2
    c = n_basis2.dag()*o_basis1
    d = n_basis2.dag()*o_basis2
    return Qobj([[a,b],[c,d]])
```

0.4.2 4.11: Express $\hat{R}_p(\theta)$ in ± 45 basis

```
def Rp(theta):
    return Qobj([[cos(theta), -sin(theta)],[sin(theta),cos(theta)]]).tidyup()

Shv45 = sim_transform(H,V,P45,M45)
```

0.4.3 4.12:

```
Rp45 = Shv45*Rp(pi/4)*Shv45.dag()

Rp45*Shv45*P45 == Shv45*V   # convert P45 to the \pm45 basis

Rp45* Qobj([[1],[0]])

ShvLR = sim_transform(H,V,L,R)

ShvLR*Rp(pi/4)*ShvLR.dag()
```

End

0.5 Chapter 6 - Spin

A few new operators (or new names for the same ones!) The three axes, x, y, z spin components can be measured with SA_x, SA_y, and SA_z devices.

We'll use $\hbar = 1$ for numerical results, this is fairly standard practice, but can be tricky to remember.

```
from numpy import sin,cos,pi,sqrt
from qutip import *

pz = Qobj([[1],[0]])
mz = Qobj([[0],[1]])
px = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
mx = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
py = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])
my = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
Sx = 1/2.0*sigmax()
Sy = 1/2.0*sigmay()
Sz = 1/2.0*sigmaz()

py
```

0.5.1 Example: determine $P(S_x = \frac{\hbar}{2}|| - y\rangle)$

```
abs(px.dag() * my) ** 2
```

0.5.2 Example: verify the commutation relation: $\left[\hat{S}_x, \hat{S}_z\right] = -i\hbar\hat{S}_y$

```
Sx*Sz - Sz*Sx == -1j*Sy  # remember, h = 1
```

0.5.3 Ex: find $\langle\hat{S}_x\rangle$ for the state $|\psi\rangle = |+Z\rangle$.

```
pz.dag()*Sx*pz
```

This makes sense given that S_x can be either $\frac{+\hbar}{2}$ or $\frac{-\hbar}{2}$ with equal probability. Similarly, if the state is $|\psi\rangle = |+x\rangle$.

```
px.dag()*Sx*px
```

Again, in units of $\hbar$.

0.6 Chapter 7: Angular Momentum

We'll use $\hbar = 1$ for numerical results, this is fairly standard practice, but can be tricky to remember.

```
from numpy import sin,cos,pi,sqrt
from qutip import *

pz = Qobj([[1],[0]])
mz = Qobj([[0],[1]])
px = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
mx = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
py = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])
my = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
Sx = 1/2.0*sigmax()
Sy = 1/2.0*sigmay()
Sz = 1/2.0*sigmaz()
Splus = Sx + 1j*Sy
Sminus = Sx - 1j*Sy

Splus*mz

Splus*pz

Sminus*pz

Sminus*mz

Splus

Sminus
```

0.7 Chapter 7 - Angular Momentum and Rotation

We'll use $\hbar = 1$ for numerical results, this is standard practice, but it is important to remember.

```
from numpy import sqrt, pi, exp, angle
from qutip import *
```

Our usual spin operators (spin-1/2)

```
pz = basis(2,0)
mz = basis(2,1)
px = 1/sqrt(2)*(pz + mz)
mx = 1/sqrt(2)*(pz - mz)
py = 1/sqrt(2)*(pz + 1j*mz)
my = 1/sqrt(2)*(pz - 1j*mz)
Sx = 1/2.0*sigmax()
Sy = 1/2.0*sigmay()
Sz = 1/2.0*sigmaz()
Splus = Sx + 1j*Sy
Sminus = Sx - 1j*Sy
```

Define the spin-1 operators. We use J just to keep them apart. Could be S instead.

```
Jx = 1/sqrt(2)*Qobj([[0,1,0],[1,0,1],[0,1,0]])
Jy = 1/sqrt(2)*Qobj([[0, -1j,0],[1j,0, -1j],[0,1j,0]])
Jz = Qobj([[1,0,0],[0,0,0],[0,0, -1]])
Jplus = Jx + 1j*Jy # raising operator
Jminus = Jx - 1j*Jy # lowering operator
```

```
Jz
```

0.8 Rotations of spin-1/2

To define an exponentiated operator, we apply the `.expm()` method to the argument. So to write the rotation operator:

$$\hat{R}_z(90^\circ) = e^{-i\frac{\pi}{2}\hat{S}_z} \tag{1}$$

we do the following. It looks a little odd to read since you might expect the `exp` to come first, but looks like this:

```
Rz90 = ( -1j*pi/2*Sz).expm()
```

```
Rz90
```

```
Rz90*px
```

Doesn't look like example 7.4 because it's not simplified. How to check:

```
exp( -1j*pi/4)*py
```

Yes, this agrees.
Can also use inner product:

```
py.dag()*Rz90*px
```

```
abs(py.dag()*Rz90*px)
```

```
angle(0.707 - 0.707j) == -pi/4
```

0.9 Find the spin-1 states (the eigenstates of the corresponding matrix)

According to the postulates, the allowed values of an observable are the eigenvalues with corresponding eigenstates.

We know the matrix representation of Jx, Jy, Jz in the Z-basis so we can find all 9 states (in the Z basis). Why nine? There are three possible values $\hbar$, 0, $-\hbar$ for each of three directions.

```
yevals, (yd,y0,yu) = Jy.eigenstates()
zevals, (zd,z0,zu) = Jz.eigenstates()
xevals, (xd,x0,xu) = Jx.eigenstates()

# Fix the signs to match book:
# (if first value of eigenstate is negative, flip all signs)
states = [xd, x0, xu, yd, y0, yu, zd, z0, zu]
states = [ -s if s.full()[0][0]<0 else s for s in states]
# Python doesn't change the values in the list so we have
# to replace the old values with the new ones:
[xd, x0, xu, yd, y0, yu, zd, z0, zu] = states

xd

Rz90 = ( -1j*pi/2*Jz).expm()

Rz90*xu == -1j*yu

Rz90*xu

yd

abs(y0.dag()*Rz90*x0)
```

0.9.1 7.10 A spin-1 particle is measured to have $S_y = \hbar$. What is the probability that a subsequent measurement will yield $S_z = 0$? $S_z = \hbar$? $S_z = -\hbar$?

```
abs(z0.dag()*yu)**2

abs(zu.dag()*yu)**2

abs(zd.dag()*yu)**2
```

0.9.2 7.11 A spin-1 particle is measured to have $S_y = 0$. What is the probability that a subsequent measurement will yield $S_x = -\hbar$?

```
abs(y0.dag()*xd)**2

abs(xd.dag()*y0)**2
```

0.9.3 7.13 $|\psi\rangle = \frac{1}{3}(2|1,1\rangle - i|1,0\rangle + 2|1,-1\rangle)$ Calculate expectation values of S^2, S_y, and S_z.

```
psi = (1/3)*(2*zu -1j*z0 +2*zd)
psi

Jsquared = Jx*Jx + Jy*Jy + Jz*Jz
Jsquared
```

```
psi.dag()*Jsquared*psi

psi.dag()*Jy*psi

psi.dag()*Jz*psi
```

0.10 Chapter 8 - Two-photon States

```
from qutip import *
from numpy import sqrt
```

```
H = basis(2,0)
V = basis(2,1)
p45 = 1/sqrt(2)*(H + V)
m45 = 1/sqrt(2)*(H - V)
R = 1/sqrt(2)*(H - 1j*V)
L = 1/sqrt(2)*(H + 1j*V)
```

```
hh = tensor(H,H)
hv = tensor(H,V)
vh = tensor(V,H)
vv = tensor(V,V)
```

```
vv
```

```
# Show these are all orthogonal:
hh.dag()*hv
```

```
vh.dag()*hv
```

```
qeye(2)
```

```
# Example 8.1
Phv = H*H.dag() - V*V.dag() # polarization measurement
ps_hv = tensor(Phv,qeye(2))
pi_hv = tensor(qeye(2),Phv)
psi_hv = tensor(Phv,Phv)

print(ps_hv*tensor(V,p45) == -tensor(V,p45))

print(pi_hv*tensor(V,p45) == tensor(V,m45))

print(psi_hv*tensor(V,p45) == -tensor(V,m45))
```

```
# Example 8.2
Pvh = tensor(V,H)*tensor(V,H).dag()
expect(Pvh,tensor(R,p45))
```

```
# Example 8.3
Phi = tensor(qeye(2),H)*tensor(qeye(2),H).dag()
expect(Phi,tensor(R,p45))
```

```
# Example 8.4
state = tensor(R,p45)
expect(Pvh,state) / expect(Phi,state)
```

0.10.1 Entangled states

```
phip = 1/sqrt(2)*(tensor(H,H) + tensor(V,V))
Phs = tensor(H,qeye(2))*tensor(H,qeye(2)).dag()
Phi = tensor(qeye(2),H)*tensor(qeye(2),H).dag()
Phshi = tensor(H,H)*tensor(H,H).dag()

expect(Phshi,phip)/ expect(Phi,phip)
```

```
P_45s45i = tensor(p45,p45)*tensor(p45,p45).dag()
P_45i = tensor(qeye(2),p45)*tensor(qeye(2),p45).dag()

expect(P_45s45i,phip)/expect(P_45i,phip)
```

Delete me, this is scratch work from review session

```
psi = 1/sqrt(5)*tensor(L,R) + 2/sqrt(5)*tensor(R,L)

psi.dag()*tensor(H,H)*tensor(H,H).dag()*psi

9/20
```

0.11 Chapter 8 - Local Realism

with Alice and Bob

```
from numpy import sin,cos,pi,sqrt,angle,exp,deg2rad,arange,rad2deg
import matplotlib.pyplot as plt
from qutip import *
%matplotlib inline
```

```
H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
```

First define the projection operator for a state at angle θ:

```
def P(theta):
    """The projection operator for a state at angle theta"""
    theta_ket = cos(theta)*H + sin(theta)*V
    return theta_ket*theta_ket.dag()
```

Create the projection operators for each of the angles, two for Alice, two for Bob

```
Pa1 = P(deg2rad(19))
Pa2 = P(deg2rad( -35))
Pb1 = P(deg2rad( -19))
Pb2 = P(deg2rad(35))
```

Create the state $|\psi\rangle = \sqrt{0.2}|H,H\rangle + \sqrt{0.8}|V,V\rangle$:

```
psi=sqrt(0.2)*tensor(H,H) + sqrt(0.8)*tensor(V,V)
```

Now, find the joint probability that Alice measures A1 and Bob measures B1. We do this by finding the expectation value of the projection operator for the joint state $|\theta_{A1},\theta_{B1}\rangle$. This is formed as the tensor product of the two appropriate projection operators. **In these tensor products, be sure to put Alice's operator first, then Bob's (just like we did for the signal and idler photons).** Each operator acts on the photon corresponding to the order in the `tensor()` function.

Notice we'll be using a new function `expect()`. This is equivalent to putting the operator in between the state bra and ket:

```
P1 = expect(tensor(Pa1,Pb1),psi)  # joint for A1, B1 (expect 0.09)
P2 = psi.dag()*tensor(Pa1,Pb1)*psi
P1 == P2
```

```
P1
```

Find the conditional probability $P(\theta_{B2}|\theta_{A1}) = \frac{P(\theta_{B2},\theta_{A1})}{P(\theta_{A1})}$

```
# B2 conditioned on A1 (expect 1)
Prob_b2_a1 = expect(tensor(Pa1,Pb2),psi)
#(psi.dag()*tensor(Pa1,Pb2)*psi).data[0,0]   # the joint probability
Prob_a1 = expect(tensor(Pa1,qeye(2)),psi)
#(psi.dag()*tensor(Pa1,qeye(2))*psi).data[0,0]  # the singular probability
Prob_b2a1 = Prob_b2_a1 / Prob_a1  # the conditional probability
Prob_b2a1
```

Find the conditional probability $P(\theta_{A2}|\theta_{B1}) = \frac{P(\theta_{A2},\theta_{B1})}{P(\theta_{B1})}$

```
# A2 conditioned on B1 (expect 1)
# can do it all on one line:
expect(tensor(Pa2,Pb1),psi) / expect(tensor(qeye(2),Pb1),psi)
```

```
expect(tensor(Pa2,Pb2),psi)  # joint for A2, B2 (classically expect 0.09, QM says 0)
```

This is what we described in class.

What if the state was just $|H,H\rangle$?

```
psi2=tensor(H,H)

expect(tensor(Pa1,Pb1),psi2)  # joint for A1, B1 (expect 0.09)

# B2 conditioned on A1:
expect(tensor(Pa1,Pb2),psi2) / expect(tensor(Pa1,qeye(2)),psi2)

# A2 conditioned on B1
expect(tensor(Pa2,Pb1),psi2) / expect(tensor(qeye(2),Pb1),psi2)

# joint for A2, B2
expect(tensor(Pa2,Pb2),psi2)
```

This is harder to interpret, but we clearly have different probabilities. Finally, check if we had used a mixed state:

A mixed state instead of the pure (entangled state).

Here we have to use the density matrix (since a `ket` cannot describe a mixed state). First some background: QuTiP has a function that gives the density matrix from a `ket` state: `ket2dm`.

```
rho_mix = 0.2 * ket2dm(tensor(H,H)) + 0.8 * ket2dm(tensor(V,V))
rho_mix

# joint for A1, B1
expect(tensor(Pa1,Pb1),rho_mix)

# B2 conditioned on A1
expect(tensor(Pa1,Pb2),rho_mix) / expect(tensor(Pa1,qeye(2)),rho_mix)

# A2 conditioned on B1
expect(tensor(Pa2,Pb1),rho_mix) / expect(tensor(Pb1,qeye(2)),rho_mix)

# joint for A2, B2:
expect(tensor(Pa2,Pb2),rho_mix)
```

We see that $P(\theta_{B2},\theta_{A2}) > P(\theta_{B1},\theta_{A1})$ as we said in class for a state that obeys realism.

Now repeat with the pure state but using density matrix techniques.

This isn't going to tell us anything new, but it shows how to work with the density matrix if you already know the `ket` state.

```
rho_pure = ket2dm(psi)  # convert from a ket to a density matrix (dm)
rho_pure
```

The calculations are actually the same in QuTiP, the `expect` function takes either a `ket` state or a density matrix.

```
# joint for A1, B1
expect(tensor(Pa1,Pb1),rho_pure)

# B2 conditioned on A1
expect(tensor(Pa1,Pb2),rho_pure) / expect(tensor(Pa1,qeye(2)),rho_pure)

# A2 conditioned on B1
expect(tensor(Pa2,Pb1),rho_pure) / expect(tensor(Pb1,qeye(2)),rho_pure)

# joint for A2, B2:
expect(tensor(Pa2,Pb2),rho_pure)
```

These all agree (as they should).

Explore the angles in more detail:

Why these angles, 19 and 35?

```
psi=sqrt(0.2)*tensor(H,H) + sqrt(0.8)*tensor(V,V)

angles = arange(1,90,1)

rads = deg2rad(angles)
```

Make a list of the probability of joint measurements for a pair of angles:

```
out = []
for r in rads:
    out.append(expect(tensor(P( -r),P(r)),psi))

plt.plot(angles,out,".") # plot in units of pi
```

We see that the joint probabilities have a zero at 35 degrees. Now plug that in to one of the conditional probabilities and see what angle for the conditional probability gives 1:

```
out = []
for r in rads:
    out.append(expect(tensor(P(r),P(deg2rad(35))),psi) / expect(tensor(P(r),qeye(2)),psi))

plt.plot(angles,out,".")
```

So only 19 and 35 work. Now, can you derive 19 and 35 given only the state $|\psi\rangle$? Try the first plot, i.e. calculate the joint probability $P(\theta_A, \theta_B)$

0.11.1 Solution

Using the state, write the projection operators for a two photon state with angles θ_A and θ_B. First, recall

$$|\theta_i\rangle = \cos\theta_i|H\rangle + \sin\theta_i|V\rangle. \tag{2}$$

Next, form the two-photon state:

$$|\theta_A, \theta_B\rangle = |\theta_A\rangle \otimes |\theta_B\rangle = \left(\cos\theta_A|H\rangle + \sin\theta_A|V\rangle\right) \otimes \left(\cos\theta_B|H\rangle + \sin\theta_B|V\rangle\right) \tag{3}$$

which we can reduce to:

$$= \cos\theta_A\cos\theta_B|H,H\rangle + \cos\theta_A\sin\theta_B|H,V\rangle + \sin\theta_A\cos\theta_B|V,H\rangle + \sin\theta_A\sin\theta_B|V,V\rangle. \tag{4}$$

Find the probability of a joint measurement of polarizations θ_A and θ_B:

$$P(\theta_A, \theta_B) = \left|\langle\psi|\theta_A, \theta_B\rangle\right|^2 \tag{5}$$

Since $|\psi\rangle$ only has $|H,H\rangle$ and $|V,V\rangle$ terms, this probability only has two terms:

$$P(\theta_A, \theta_B) = \left|\sqrt{0.2}\cos\theta_A\cos\theta_B + \sqrt{0.8}\sin\theta_A\sin\theta_B\right|^2 \tag{6}$$

Plot is shown below for $\theta_A = -\theta_B$ and it agrees perfectly with our model above.

```
# Solution:
# For the first plot, we can show the joint probability for two angles is given by:

plt.plot(rad2deg(rads),(sqrt(0.2)*cos( -rads)*cos(rads) + sqrt(0.8)*sin( -rads)*sin(rads))**2)
```

Challenge:

If we change the state to $|\psi\rangle = \sqrt{0.8}|H,H\rangle + \sqrt{0.2}|V,V\rangle$, the two angles that work for this state.

```
# Solution

psi3=sqrt(0.8)*tensor(H,H) + sqrt(0.2)*tensor(V,V)

out = []
for r in rads:
    out.append(expect(tensor(P( -r),P(r)),psi3))

plt.plot(angles,out,".") # plot in units of pi

# Solution
out = []
for r in rads:
    out.append(expect(tensor(P(r),P(deg2rad(55))),psi3) / expect(tensor(P(r),qeye(2)),psi3))

plt.plot(angles,out,".")
```

0.12 Chapter 8 - Density Matrix

```
from numpy import sqrt
from qutip import *

H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
```

0.12.1 Example 8.A.1:

```
psi = 1/sqrt(2)*tensor(H,H) + 1/sqrt(2)*tensor(V,V)
psi

rho_ent = psi*psi.dag()

(rho_ent*rho_ent).tr()

rho_mix = 0.5*tensor(H,H)*tensor(H,H).dag() + 0.5*tensor(V,V)*tensor(V,V).dag()

(rho_mix*rho_mix).tr()
```

0.12.2 Example 8.A.2

Remember the 45 states:

```
P45 = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
M45 = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
```

Create the projection operator for $|+45, +45\rangle$

```
Proj_4545 = tensor(P45,P45)*tensor(P45,P45).dag()

(Proj_4545*rho_mix).tr()
```

Create projection operator for $|+45\rangle_i$

```
Proj_45i = tensor(qeye(2),P45)*tensor(qeye(2),P45).dag()

(Proj_45i*rho_mix).tr()

0.25/0.5
```

Extend the example for the pure (superposition) state

```
(Proj_4545*rho_ent).tr() / (Proj_45i*rho_ent).tr()
```

The photons are entanlged and therefore show perfect correlation even in the +45 measurements.

0.13 Chapter 10 - Position and Momentum

We can start using sympy to handle symbolic math (integrals and other calculus):

```
from sympy import *

init_printing(use_unicode=True)

# SymPy works better if you specify what letters are symbols:
x, y, z = symbols('x y z', real=True)

# notice we can also put some restrictions on the symbols:
a, c = symbols('a c', nonzero=True, real=True)

integrate?
```

There are two ways to use the `integrate` function. In one line, like `integrate(x,(x,0,1))` or by naming an expression and then integrating it over a range:

```
A = (c*cos((pi*x)/(2.0*a)))**2
A.integrate((x, -a,a),conds='none')
```

We'll use both, at different times. For longer expressions, the second form can be easier to read and write.

First, just try the following, then we'll re-create some examples in the book.

```
integrate(x,(x,0,1))

integrate(x**2,(x,0,1))

A = (c*cos((pi*x)/(2.0*a)))**2
A.integrate((x, -a,a))
```

So this tells us the normalization constant should be $c = \frac{1}{\sqrt{a}}$. Check that it is normalized if we do that:

```
psi = 1/sqrt(a)*cos((pi*x)/(2.0*a))  # notice we can name the expression something useful.
B = psi**2
B.integrate( (x, -a,a), conds='none')
```

Because `psi` is a real function, we can calculate expectation values by integrating over x or x^2 with `psi**2`:

```
C = x*psi**2
C.integrate( (x, -a,a), conds='none')

D = x**2 * psi**2
E = D.integrate( (x, -a,a), conds='none')
E

E.simplify()  # this is a useful method!

E.n()  # the .n() method approximates the numerical part. You can look at the full expression
```

0.13.1 Example 10.2

```
h = Symbol('hbar', real=True, positive=True)
```

Use the `diff` function to take a derivative of a symbolic expression. For example:

```
diff(x**2, x)

# Solution
 -1j*h*diff( 1/a*cos((pi*x)/(2*a)) ,x)

# Solution
B1 = (pi*h/(2*a))**2 * (cos((pi*x)/(2*a)))**2
B1.integrate( (x, -a,a), conds='none' )
```

0.13.2 Example 10.3

```
p = Symbol('p', real=True)

# Solution
A = integrate(1/sqrt(2*pi*a*h)*exp( -I*p*x/h)*cos((pi*x)/(2*a)),(x, -a,a), conds='none')

# Solution
A

psi_p = sqrt(2*a*pi/h) * 2/(pi**2 - (2*p*a/h)**2) * cos(p*a/h)
psi_p

psi_p == sqrt(2*a*pi/h)*2/(pi**2 - (2*p*a/h)**2) * cos(p*a/h)
```

Which agrees with the book.

This is about as far as we can go in sympy. Unfortunately, many other momentum integrals choke. There are a few hints to get through the rest here:

0.13.3 Problem 10.3

```
x, y, z = symbols('x y z', real=True)
a, c = symbols('a c', nonzero=True, real=True, positive=True)

psi = c*1/(a**2 + x**2)  # define the wavefunction with c constant
int1 = integrate(psi*psi,(x, -oo,oo), conds='none')  # integrate psi^2
solutions = solve(int1 - 1,c)  # solve for c, this returns a list of solutions
c2 = simplify(solutions[0])  # simplify the solution for c:
c2

psi2 = c2/c*psi
psi2

integrate(psi2 * x * psi2,(x, -oo,oo))

integrate(psi2 * x**2 * psi2,(x, -oo,oo))
```

So $\Delta x^2 = a^2 - 0^2$ therefore $\Delta x = a$

0.13.4 Problem 10.17:

Now find the momentum representation of the state from 10.3

```
p = symbols('p', nonzero=True, real=True, positive=True)
B = integrate(sqrt(1/(2*pi*h))*exp( -I*p*x/h)*psi2,(x, -oo,oo))
B

B.simplify()
```

This agrees with the book after we notice that we had to force p to be positive in order to get the integral to converge. The book has $|p|$ in the argument of the exponent to reflect this constraint.

0.13.5 Problem 10.13

```
psi = 1/sqrt(2*a)*sech(x/a)
dpsi = diff(psi,x)
dpsi

ddpsi = diff(dpsi,x)
ddpsi = ddpsi.simplify()
ddpsi

expect_p = integrate(psi*ddpsi,(x, -oo,oo))
expect_p
```

0.14 Chapter 10 - Position and Momentum

We can start using sympy to handle symbolic math (integrals and other calculus):

```
from sympy import *

init_printing(use_unicode=True)

x, y, z = symbols('x y z', real=True)
a, c = symbols('a c', nonzero=True, real=True)

integrate?
```

There are two ways to use the `integrate` function. In one line, like `integrate(x,(x,0,1))` or by naming an expression and then integrating it over a range:

```
A = (c*cos((pi*x)/(2.0*a)))**2
A.integrate((x, -a,a),conds='none')
```

We'll use both, at different times. For longer expressions, the second form can be easier to read and write.

First, just try the following, then we'll re-create some examples in the book.

```
integrate(x,(x,0,1))

integrate(x**2,(x,0,1))
```

The cell below will return an odd set of conditions on the result. This is because the solver doesn't want to assume anything about `a` and there is a special case where the answer would be different. If you look closely though, that special case isn't physically realistic so to igore these special conditions, we add `conds='none'`. The next cell down does what you'd expect. From here on out, just add this to the `integrate` function and we'll get what we expect.

```
A = (c*cos((pi*x)/(2.0*a)))**2
A.integrate((x, -a,a))

A = (c*cos((pi*x)/(2.0*a)))**2
A.integrate((x, -a,a), conds='none')
```

So this tells us the normalization constant should be $c = \frac{1}{\sqrt{a}}$. Check that it is normalized if we do that:

```
psi = 1/sqrt(a)*cos((pi*x)/(2.0*a))  # notice we can name the expression something useful.
B = psi**2
B.integrate( (x, -a,a), conds='none')
```

Because `psi` is a real function, we can calculate expectation values by integrating over x or x^2 with `psi**2`:

```
C = x*psi**2
C.integrate( (x, -a,a), conds='none')

D = x**2 * psi**2
E = D.integrate( (x, -a,a), conds='none')

E.n()  # the .n() method approximates the numerical part. You can look at the full expression

E
```

0.14.1 Example 10.2

```
h = Symbol('hbar', real=True)
```

Use the `diff` function to take a derivative of a symbolic expression. For example:

```
diff(x**2, x)
```

```
# Solution goes here
```

```
# Solution goes here
```

0.14.2 Example 10.3

```
p = Symbol('p', real=True)
```

```
# Solution goes here
```

```
# Solution goes here
```

0.15 Chapter 12 - Simple Harmonic Oscillator

Several examples that make use of the built-in Simple Harmonic Oscillator tools:

```
from numpy import sqrt,linspace,exp
from qutip import *
import matplotlib.pyplot as plt
%matplotlib inline

N = 40 # size of the Hilbert space.
       # N needs to be large enough that matrix is not truncated
a = destroy(N)
n = a.dag()*a
```

Define a coherent state with $\alpha = 2$:

```
psi = coherent(N,1)

psi
```

Expectation value $\langle n \rangle$:

```
psi.dag()*n*psi

# Convert to a density matrix:
psi_dm = ket2dm(psi)

psi*psi.dag()

# Fock state distribution of the coherent state:
plot_fock_distribution(psi_dm)
```

Also look at other coherent states:

```
# With larger N and \alpha:
plot_fock_distribution(coherent_dm(100,6))
```

And some other density matrices, like a fock-state (or n eigenstate):

```
# With a Fock state (number state):
plot_fock_distribution(fock_dm(20,2))

fock(20,2)
```

And a thermal state:

```
# A thermal state has decaying amplitudes
plot_fock_distribution(thermal_dm(20,2))
```

0.15.1 Finally, can visualize the phasor using what is called the Wigner function:

```
xvec = linspace( -10,10,200) # Create an array for the phase space coordinates.
plt.imshow(wigner(coherent(40,5j)+coherent(40, -5j),xvec,xvec), extent=[ -10,10, -10,10]) # co
```

0.16 Chapter 13 - Hydrogen functions

Start with some imports fro Symbolic Python library:

```
from sympy.physics.hydrogen import R_nl
from sympy.functions.special.spherical_harmonics import Ynm
from sympy import *
```

Define some variables, radial, polar, azimuthal, time, and two frequencies:

```
var("r theta phi t w1 w2")
```

0.16.1 Look at a few of the radial equations and the spherical harmonics

Notice that instead of Ylm the name is Ynm... the arguments to the function are still the quantum numbers `l` and `m`

```
R_nl(1, 0, r, 1)  # the n = 1, l = 0 radial function
```

```
Ynm(0,0,theta,phi).expand(func=True)  # the l = 0, m = 0 spherical harmonic
```

Write the equation for the $|nlm\rangle = |100\rangle$ state. Use the sympy method `.expand(func=True)` to convert to the actual expression. To create this state, we combine the Radial function and the Ylm function. Make sure to set n, l, and m to the correct values. The fourth argument to `R_nl` is `Z` which we set to 1 since we are talking about a 1-proton nucleus.

The combination of R_nl and Ynm should look like the following (replace N, L, and M with the appropriate values):

```
R_nl(N, L, r, 1)*Ynm(L, M, theta, phi).expand(func=True)
```

```
# this is the |100> state:
psi100 = R_nl(1, 0, r, 1)*Ynm(0,0,theta,phi).expand(func=True)
```

```
psi100  # check to see how it looks as an expression
```

0.16.2 Integrating over all space

Remember spherical coordinate integrals of function $f(r, \theta, \phi)$ over all space look like:

$$\int_0^\infty \int_0^\pi \int_0^{2\pi} r^2 \sin(\theta) dr d\theta d\phi \; f(r, \theta, \phi) \tag{7}$$

so you alwasy need to add a factor of `r**2*sin(theta)` and then integrate `r` from 0 to infinity, `theta` from $0 - \pi$ and `phi` from $0 - 2\pi$. As a check, you should integrate the square of the `psi100` wavefunction over all space to see that it equals 1 (i.e. it is normalized)

```
integrate(r**2*sin(theta) * (psi100)**2 ,(r,0,oo),(theta,0,pi),(phi,0,2*pi))
```

0.16.3 Now do the $|210\rangle$ state:

```
psi210 = R_nl(2, 1, r, 1)*Ynm(1,0,theta,phi).expand(func=True)
```

```
psi210  # check how it looks
```

Note, if you compare these to listed solutions (for example at http://hyperphysics.phy-astr.gsu.edu/hbase/quantum/hydwf.html#c3) you see that there are not any factors of a_0. This is because the `R_nl` function is defined in units of a_0. a_0 is the Bohr Radius: Bohr radius

```
psi211 = R_nl(2, 1, r, 1)*Ynm(1,1,theta,phi).expand(func=True)
psi211
```

0.16.4 Now calculate $\langle z \rangle$:

To calculate $\langle z \rangle$ we need to convert to spherical coordinates: $z = r\cos\theta$. The terms in the following integral are the $r^2 \sin\theta$ then z (in spherical coords) then the wave function squared.

```
expect = integrate(r**2*sin(theta)* (r*cos(theta)) * (psi100*psi100),(r,0,oo),(theta,0,pi),(ph

expect
```

No surprise, the average z position of the electron in the hydrogen atom is 0.

0.16.5 Now for problem 13.21

find $\langle z \rangle(t)$. Use the same integral, but add a time-dependent piece to each term in the wavefunction, add them together and multiply by the complex conjugate.

```
psi = 1/sqrt(2)*(psi100*exp(1j*w1*t) + psi210*exp(1j*w2*t))
psi_conj = 1/sqrt(2)*(psi100*exp( -1j*w1*t) + psi210*exp( -1j*w2*t))

outer = (psi*psi_conj).simplify()

outer

expect2 = integrate(r**2 * sin(theta) * (r*cos(theta)) * outer,(r,0,oo),(theta,0,pi),(phi,0,2*

expect2

expect2.simplify()
```

We need to interpret this result. First you should show that this expression is simply a constant amplitude factor times $\cos((w2 - w1)t)$, in other words $\langle z \rangle$ oscillates at frequency `w2-w1`.

0.16.6 Your assignment:

Explore other combinations of states and draw conclusions about the z behavior from the results. You may not be able to get these expressions to simplify, but the important thing is to look for the time dependence and simplify that part.

- Does $\langle z \rangle$ oscillate for any combination of two Hydrogen states $|nlm\rangle$?
- Are there restrictions on what n values give oscillating $\langle z \rangle$ expressions? (hint, to keep it simple, always let one state be the n=1 state)
- How does $\langle z \rangle$ change with different l and m values are used in the state?

Hints for interpreting your results:

- What are the relavant frequencies in your expression for $\langle z \rangle$ and why?
- Simplify one of your $\langle z \rangle$ expressions and write the time dependence in terms of the frequencies w2 and w1.

0.17 Lab 1 - Vectors and Matrices

This notebook demonstrates the use of vectors and matrices in IPython. Note that the basis is not explicit in any of these operations. You must keep track of the basis yourself (using variable names, or notes etc).

```
from numpy import array, dot, outer, sqrt, matrix
from numpy.linalg import eig, eigvals
from matplotlib.pyplot import hist

%matplotlib inline

rv = array([1,2])  # a row vector
rv

cv = array([[3],[4]])  # a column vector
cv

rv
```

Two kinds of vector products we'll see: inner product (dot product) and outer product

1) Use the function *dot(vector1, vector2)* to find the dot product of rv and cv. Does the order of the arguments matter?

```
dot(rv,cv)

# dot(cv,rv)
```

2) Use the function *outer(vector1, vector2)* to find the outer product of rv and cv. Does the order of the arguments matter?

```
outer(rv,cv)

outer(cv,rv)
```

II. Complex vectors

```
# Complex numbers in python have a j term:
a = 1+2j
a

v1 = array([1+2j, 3+2j, 5+1j, 4+0j])
v1
```

The complex conjugate changes the sign of the imaginary part:

```
v1.conjugate()
```

3) Use *dot()* and *.conjugate()* to find the dot product of v1 and it's own conjugate:

```
dot(v1.conjugate(),v1)
```

III. Matrices

```
# a two -dimensional array
m1 = array([[0,1],[ -2,3]])
m1

# can find transpose with the T method:
m1.T

# find the eigenvalues and eigenvectors of a matrix:
eig(m1)
```

Can also use the `matrix` type which is like array but restricts to 2D. Also, `matrix` adds `.H` and `.I` methods for hermitian and inverse, respectively. For more information, see Stack Overflow question #4151128

```
m2 = matrix( [[2,1],[2,1]])

m2.H

evals, evec = eig(m2)

evals

evec

# use a question mark to get help on a command
eig?
```

0.17.1 Examples:

Example 1.4

Find the eigenvalues and eigenvectors of M = ([0,1],[-2,3]])

```
M14 = array([[0,1],[ -2,3]])

eig(M14)
```

Interpret this result: the two eigenvalues are 1 and 2 the eigenvectors are strange decimals, but we can check them against the stated solution:

```
1/sqrt(2)  # this is the value for both entries in the first eigenvector

1/sqrt(5)  # this is the first value in the second eigenvector

2/sqrt(5)  # this is the second value in the second eigenvector

eig(M14)
```

Signs are opposite compared to the book, but it turns out that (-) doesn't matter in the interpretation of eigenvectors: only "direction" matters (the relative size of the entries).

Example: Problem 1.16 using Python functions

```
M16 = array([[0, -1j],[1j,0]])

evals, evecs = eig(M16)

evecs

evecs[:,0]

evecs[:,1]

dot(evecs[:,0].conjugate(),evecs[:,1])
```

0.17.2 Part 2: Using QuTiP

Keeping track of row and column vectors in Ipython is somewhat artificial and tedious. The QuTiP library is designed to take care of many of these headaches

```
from qutip import *

# Create a row vector:
qv = Qobj([[1,2]])
qv

# Find the corresponding column vector
qv.dag()

qv2 = Qobj([[1+2j,4 -1j]])
qv2

qv2.dag()
```

Vector products in QuTiP

Only need to know one operator: "*" The product will depend on the order, either inner or outer

```
qv2*qv2.dag()  # inner product (dot product)

qv2.dag()*qv2  # outer product
```

Matrix in QuTiP

```
qm = Qobj([[1,2],[2,1]])
qm

qm.eigenenergies()  # in quantum (as we will learn) eigenvalues often correspond to energy lev

evals, evecs = qm.eigenstates()

evecs

evecs[0]
```

Example from class:

```
qv = Qobj([[0,1],[ -2,3]])
qv

evals, evecs = qv.eigenstates()

evals

evecs

qv*evecs[0]

qv*evecs[0] == evals[0]*evecs[0]

qv*evecs[1] == evals[1]*evecs[1]
```

0.17.3 Some additional examples we have already done by hand:

Problem 1.2 using the *hist()* function.

Create a histogram of the data and from it estimate the probability distribution. Use min and max values for the range of 5 and 14, and use 5 bins (5 and 6 go in the first bin).

```
# Solution
data = [10,13,14,14,6,8,7,9,12,14,13,11,10,7,7]
# Fill in the hist() function:
n, bins, patches = hist(data, bins=9, range=(5,14))
```

Problem 1.8

Find the constant c that normalizes the probability density:

$$p(x) = \begin{cases} ce^{-ax}, & x \geq 0 \\ \\ 0 & x < 0 \end{cases} \tag{8}$$

Hint: using sympy, we can calculate the relevant integral. The conds='none' asks the solver to ignore any strange conditions on the variables in the integral. This is fine for most of our integrals. Usually the variables are real and well-behaved numbers.

```
# Partial Solution:
from sympy import *
c,a,x = symbols("c a x")
Q.positive((c,a))
first = integrate( c*exp( -a*x) ,(x,0,oo),conds='none')
first

check = integrate( a*exp( -a*x) ,(x,0,oo),conds='none')
check
```

0.18 Lab 1 - Vectors and Matrices

This notebook demonstrates the use of vectors and matrices in IPython. Note that the basis is not explicit in any of these operations. You must keep track of the basis yourself (using variable names, or notes etc).

```
from numpy import array, dot, outer, sqrt, matrix
from numpy.linalg import eig, eigvals
from matplotlib.pyplot import hist

%matplotlib inline

rv = array([1,2])  # a row vector
rv

cv = array([[3],[4]])  # a column vector
cv
```

Two kinds of vector products we'll see: inner product (dot product) and outer product

1) Use the function *dot(vector1, vector2)* to find the dot product of rv and cv. Does the order of the arguments matter?

```
dot(rv,cv)

# uncomment and run this line, it should give an error, make sure you understand why
# dot(cv,rv)
```

2) Use the function *outer(vector1, vector2)* to find the outer product of rv and cv. Does the order of the arguments matter?

```
outer(rv,cv)

outer(cv,rv)
```

II. Complex vectors

```
# Complex numbers in python have a j term:
a = 1+2j

v1 = array([1+2j, 3+2j, 5+1j, 4+0j])
```

The complex conjugate changes the sign of the imaginary part:

```
v1.conjugate()
```

3) Use *dot()* and *.conjugate()* to find the dot product of v1 and it's own conjugate:

```
dot(v1.conjugate(),v1)
```

III. Matrices

```
# a two -dimensional array
m1 = array([[2,1],[2,1]])
m1

# can find transpose with the T method:
m1.T

# find the eigenvalues and eigenvectors of a matrix:
eig(m1)
```

Can also use the `matrix` type which is like array but restricts to 2D. Also, `matrix` adds `.H` and `.I` methods for hermitian and inverse, respectively. For more information, see Stack Overflow question #4151128

```
m2 = matrix( [[2,1],[2,1]])

m2.H

eig(m2)

# use a question mark to get help on a command
eig?
```

0.18.1 Examples:

Example 1.4

Find the eigenvalues and eigenvectors of M = ([0,1],[-2,3]])

```
M14 = array([[0,1],[ -2,3]])

eig(M14)
```

Interpret this result: the two eigenvalues are 1 and 2 the eigenvectors are strange decimals, but we can check them against the stated solution:

```
1/sqrt(2)  # this is the value for both entries in the first eigenvector

1/sqrt(5)  # this is the first value in the second eigenvector

2/sqrt(5)  # this is the second value in the second eigenvector

eigvals(M14)
```

Signs are opposite compared to the book, but it turns out that (-) doesn't matter in the interpretation of eigenvectors: only "direction" matters (the relative size of the entries).

Example: Problem 1.16 using Ipython functions

```
M16 = array([[0, -1j],[1j,0]])

evals, evecs = eig(M16)

evecs

evecs[:,0]

evecs[:,1]

dot(evecs[:,0].conjugate(),evecs[:,1])
```

0.18.2 Part 2: Using QuTiP

Keeping track of row and column vectors in Ipython is somewhat artificial and tedious. The QuTiP library is designed to take care of many of these headaches

```
from qutip import *

# Create a row vector:
qv = Qobj([[1,2]])
qv

# Find the corresponding column vector
qv.dag()

qv2 = Qobj([[1+2j,4 -1j]])
qv2

qv2.dag()
```

Vector products in QuTiP

Only need to know one operator: "*" The product will depend on the order, either inner or outer

```
qv2*qv2.dag()  # inner product (dot product)

qv2.dag()*qv2  # outer product
```

Matrix in QuTiP

```
qm = Qobj([[1,2],[2,1]])
qm

qm.eigenenergies()  # in quantum (as we will learn) eigenvalues often correspond to energy lev

evals, evecs = qm.eigenstates()

evecs

evecs[0]
```

0.18.3 Practice:

Problem 1.2 using the *hist()* function.

```
# Solution
n, bins, patches = hist([10,13,14,14,6,8,7,9,12,14,13,11,10,7,7],bins=5,range=(5,14))

# Solution
n

# Solution
pvals = n/n.sum()
```

Problem 1.8

Hint: using sympy, we can calculate the relevant integral. The conds='none' asks the solver to ignore any strange conditions on the variables in the integral. This is fine for most of our integrals. Usually the variables are real and well-behaved numbers.

```
# Solution
from sympy import *
c,a,x = symbols("c a x")
Q.positive((c,a))
first = integrate(c*exp( -a*x),(x,0,oo),conds='none')
print("first = ",first)
second = integrate(a*exp( -a*x),(x,0,oo),conds='none')
print("second = ",second)
```

0.19 Lab 2 - Quantum States

Useful for working examples and problems with photon quantum states. You may notice some similarity to the Jones Calculus ;-)

```
import numpy as np
from qutip import *
```

These are the polarization states:

```
H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
P45 = Qobj([[1/np.sqrt(2)],[1/np.sqrt(2)]])
M45 = Qobj([[1/np.sqrt(2)],[ -1/np.sqrt(2)]])
R = Qobj([[1/np.sqrt(2)],[ -1j/np.sqrt(2)]])
L = Qobj([[1/np.sqrt(2)],[1j/np.sqrt(2)]])
```

```
V
```

Devices:
HWP - Half-wave plate axis at θ to the horizontal
LP - Linear polarizer, axis at θ
QWP - Quarter-wave plate, axis at θ
Note, these are functions so you need to call them with a specific value of theta.

```
def HWP(theta):
    return Qobj([[np.cos(2*theta),np.sin(2*theta)],[np.sin(2*theta), -np.cos(2*theta)]]).tidyu
```

```
def LP(theta):
    return Qobj([[np.cos(theta)**2,np.cos(theta)*np.sin(theta)],[np.sin(theta)*np.cos(theta),n
```

```
def QWP(theta):
    return Qobj([[np.cos(theta)**2 + 1j*np.sin(theta)**2,
                 (1 -1j)*np.sin(theta)*np.cos(theta)],
                [(1 -1j)*np.sin(theta)*np.cos(theta),
                 np.sin(theta)**2 + 1j*np.cos(theta)**2]]).tidyup()
```

```
QWP(np.pi/4)
```

0.19.1 Example 1) Check that the $|H\rangle$ state is normalized

```
H.dag()*H
```

To show more information on an object, use the question mark after the function or object:

```
np.sin?
```

Example 2) Converting from ket to bra:

```
psi = Qobj([[1+1j],[2 -1j]])
psi
```

```
psi.dag()
```

```
psi.dag().dag()
```

the `.dag()` python method computes the "daggar" or the complex transpose.

0.19.2 1) Is `psi` normalized? If not, find the normalization constant and confirm that constant normalizes `psi`.

0.19.3 2) Verify that the $|V\rangle$ state is normalized

0.19.4 3) Verify that the $|H\rangle$ and $|V\rangle$ states are orthogonal. Repeat for the other pairs of states.

0.19.5 4) Calculate the horizontal component c_H of the state $\psi = \frac{1}{\sqrt{5}}|H\rangle + \frac{2}{\sqrt{5}}|V\rangle$

0.19.6 5) Verify Eq. (3.18), $P(H||45\rangle) = \frac{1}{2}$, (which states "The probability that a photon prepared in the +45 state will leave a PA_HV in the Horizontal state is one half.")

0.19.7 6) Demonstrate that a half-wave plate at 45-degrees converts $|H\rangle$ to $|V\rangle$

0.19.8 7) Re-create Figure 3.9 by plotting the probability P(+45) vs phase φ

0.20 Quantum states

Useful for working examples and problems with photon quantum states. You may notice some similarity to the Jones Calculus ;-)

```
from numpy import sqrt
from qutip import *
```

These are the polarization states:

```
H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
P45 = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
M45 = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
R = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
L = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])
```

Devices:
HWP - Half-wave plate axis at θ to the horizontal
LP - Linear polarizer, axis at θ
QWP - Quarter-wave plate, axis at θ
Note, these are functions so you need to call them with a specific value of theta.

```
def HWP(theta):
    return Qobj([[cos(2*theta),sin(2*theta)],[sin(2*theta), -cos(2*theta)]]).tidyup()

def LP(theta):
    return Qobj([[cos(theta)**2,cos(theta)*sin(theta)],[sin(theta)*cos(theta),sin(theta)**2]])

def QWP(theta):
    return Qobj([[cos(theta)**2 + 1j*sin(theta)**2,
                  (1 -1j)*sin(theta)*cos(theta)],
                 [(1 -1j)*sin(theta)*cos(theta),
                  sin(theta)**2 + 1j*cos(theta)**2]]).tidyup()
```

0.20.1 Example 1) Check that the $|H\rangle$ state is normalized

```
H.dag()*H
```

```
psi = Qobj([[1+1j], [2 -1j]])
psi
```

```
psi.dag()
```

```
psi.dag().dag()
```

1) verify that the $|V\rangle$ state is normalized

2) Verify that the $|H\rangle$ and $|V\rangle$ states are orthogonal. Repeat for the other pairs of states.

3) Find the horizontal component c_H of the state $\psi = \frac{1}{\sqrt{5}}|H\rangle + \frac{2}{\sqrt{5}}|V\rangle$

4) Verify Eq. (3.18)

0.21 Lab 2 - Quantum States, SOLUTIONS

Useful for working examples and problems with photon quantum states. You may notice some similarity to the Jones Calculus ;-)

```
import numpy as np
from qutip import *
```

These are the polarization states:

```
H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
P45 = Qobj([[1/np.sqrt(2)],[1/np.sqrt(2)]])
M45 = Qobj([[1/np.sqrt(2)],[ -1/np.sqrt(2)]])
R = Qobj([[1/np.sqrt(2)],[ -1j/np.sqrt(2)]])
L = Qobj([[1/np.sqrt(2)],[1j/np.sqrt(2)]])
```

```
V
```

```
Hbra = H.dag()
```

```
Hbra*V
```

Devices:
HWP - Half-wave plate axis at θ to the horizontal
LP - Linear polarizer, axis at θ
QWP - Quarter-wave plate, axis at θ
Note, these are functions so you need to call them with a specific value of theta.

```
def HWP(theta):
    return Qobj([[np.cos(2*theta),np.sin(2*theta)],[np.sin(2*theta), -np.cos(2*theta)]]).tidyu
```

```
def LP(theta):
    return Qobj([[np.cos(theta)**2,np.cos(theta)*np.sin(theta)],[np.sin(theta)*np.cos(theta),n
```

```
def QWP(theta):
    return Qobj([[np.cos(theta)**2 + 1j*np.sin(theta)**2,
                 (1 -1j)*np.sin(theta)*np.cos(theta)],
                [(1 -1j)*np.sin(theta)*np.cos(theta),
                 np.sin(theta)**2 + 1j*np.cos(theta)**2]]).tidyup()
```

```
QWP(np.pi/4)
```

0.21.1 Example 1) Check that the $|H\rangle$ state is normalized

```
H.dag()*H
```

To show more information on an object, use the question mark after the function or object:

```
np.sin?
```

Example 2) Converting from ket to bra:

```
psi = Qobj([[1+1j],[2 -1j]])
psi
```

```
psi.dag()
```

```
psi.dag().dag()
```

the `.dag()` python method computes the "daggar" or the complex transpose.

0.21.2 1) Is `psi` normalized? If not, find the normalization constant and confirm that constant normalizes `psi`.

```
psi.dag()*psi

psi_norm = psi*np.sqrt(1/7)

psi_norm.dag() * psi_norm
```

0.21.3 2) Verify that the $|V\rangle$ state is normalized

```
V.dag()*V
```

0.21.4 3) Verify that the $|H\rangle$ and $|V\rangle$ states are orthogonal. Repeat for the other pairs of states.

```
H.dag()*V

L.dag()*R

P45.dag()*M45
```

0.21.5 4) Calculate the horizontal component c_H of the state $\psi = \frac{1}{\sqrt{5}}|H\rangle + \frac{2}{\sqrt{5}}|V\rangle$

```
psi = 1/np.sqrt(5)*H + 2/np.sqrt(5)*V
psi

psi2 = Qobj([[1/np.sqrt(5)],[2/np.sqrt(5)]])
psi2

H.dag()*psi
```

0.21.6 5) Verify Eq. (3.18), $P(H||45\rangle) = \frac{1}{2}$, (which states "The probability that a photon prepared in the +45 state will leave a PA_HV in the Horizontal state is one half.")

```
expect(H*H.dag(),P45)

np.conjugate(H.dag()*P45) * (H.dag()*P45)
```

0.21.7 6) Demonstrate that a half-wave plate at 45-degrees converts $|H\rangle$ to $|V\rangle$

```
HWP(np.radians(45)) * H

HWP(np.pi/4) * H == V
```

0.21.8 7) Re-create Figure 3.9 by plotting the probability P(+45) vs phase φ

```
import matplotlib.pyplot as plt

plt.plot([1,2,3,2,3,4])

phi_list = np.linspace(0,8*np.pi,num=100)

plt.plot(phi_list,np.sin(phi_list))

sin_list = [np.sin(p) for p in phi_list] # notes
```

```
plt.plot(phi_list,sin_list)
```

```
def psi(phi):
    return 1/np.sqrt(2)*(H + np.exp(1j*phi)*V)

answer = [expect(M45*M45.dag(),psi(phi)) for phi in phi_list]

plt.plot(phi_list/np.pi,answer," -o")
plt.xlabel("$\\phi$")
plt.ylabel("P( -45)")
```

0.22 Lab 3: Operators

An overview of operator properties

```
import matplotlib.pyplot as plt
from numpy import sqrt,cos,sin,arange,pi
from qutip import *
%matplotlib inline
```

```
H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
P45 = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
M45 = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
R = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
L = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])
```

0.22.1 Example 1: the outer product and the projection operator

We already have the $|H\rangle$ state represented as a vector in the HV basis, so the $\hat{P}_H$ operator is the outer product $|H\rangle\langle H|$ (a ket then a bra):

```
H
```

```
Ph = H*H.dag()
Ph
```

Same with the $\hat{P}_V$ operator:

```
Pv = V*V.dag()
Pv
```

0.22.2 Example 2: Verify Eq. 4.38 for the HV basis states. Repeat for the ±45, and LR basis

```
identity(2)
```

```
Ph + Pv == identity(2)
```

```
P45*P45.dag()
```

```
M45*M45.dag()
```

```
P45*P45.dag() + M45*M45.dag()
```

```
L*L.dag()
```

```
R*R.dag()
```

```
L*L.dag() + R*R.dag()
```

0.22.3 Example 3: Represent the $\hat{R}_p(\theta)$ operator in the HV basis and verify your representation by operating on $|H\rangle$ and $|V\rangle$ states. Use the following template function definition.

```
def Rp(theta):
    return Qobj([[cos(theta), -sin(theta)],[sin(theta),cos(theta)]]).tidyup()
```

```
Rp(pi/2)
```

```
V==Rp(pi/2)*H
```

```
# Solution Goes Here
```

0.22.4 1) Using the $\hat{R}_p(\theta)$ operator, verify the operator properties described in Sections 4.1 and 4.2. Specifically, verify Eqns. 4.6, 4.7, 4.16, 4.18, 4.22, and 4.27

```
# Solution Goes Here
```

0.22.5 Example: the similarity transform

The following defines a function that creates a similarity transform matrix. It takes the two old basis vectors and the two new basis vectors as arguments. To apply the transform, simply multiply the matrix onto the state vector or ooperator matrix. Following the examples below, explore this transform.

```
def sim_transform(o_basis1, o_basis2, n_basis1, n_basis2):
    a = n_basis1.dag()*o_basis1
    b = n_basis1.dag()*o_basis2
    c = n_basis2.dag()*o_basis1
    d = n_basis2.dag()*o_basis2
    return Qobj([[a,b],[c,d]])
```

We can define a similarity transform that converts from $HV \rightarrow \pm 45$

```
Shv45 = sim_transform(H,V,P45,M45)  # as found in Example 4.A.1, Eq. 4.A.10.
Shv45
```

```
Shv45 * H  # compare to Eq. 4.A.12
```

0.22.6 4) Use the similarity transform to represent $|V\rangle$ in the ± 45 basis

0.22.7 5) Represent $\hat{P}_H$ in the ± 45 basis.

Check your answer against Eqns. 4.A.17 and 4.72

0.22.8 6) Represent $\hat{P}_V$ in the ± 45 basis.

0.23 Lab 3: Operators

An overview of operator properties

```
import matplotlib.pyplot as plt
from numpy import sqrt,cos,sin,arange,pi
from qutip import *
%matplotlib inline
```

```
H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
P45 = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
M45 = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
R = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
L = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])
```

0.23.1 Example 1: the outer product and the projection operator

We already have the $|H\rangle$ state represented as a vector in the HV basis, so the $\hat{P}_H$ operator is the outer product $|H\rangle\langle H|$ (a ket then a bra):

```
H
```

```
Ph = H*H.dag()
Ph
```

Same with the $\hat{P}_V$ operator:

```
Pv = V*V.dag()
Pv
```

0.23.2 Example 2: Verify Eq. 4.38 for the HV basis states. Repeat for the ±45, and LR basis

```
identity(2)
```

```
Ph + Pv == identity(2)
```

```
P45*P45.dag()
```

```
M45*M45.dag()
```

```
P45*P45.dag() + M45*M45.dag()
```

```
L*L.dag()
```

```
R*R.dag()
```

```
L*L.dag() + R*R.dag()
```

0.23.3 Example 3: Represent the $\hat{R}_p(\theta)$ operator in the HV basis and verify your representation by operating on $|H\rangle$ and $|V\rangle$ states. Use the following template function definition.

```
def Rp(theta):
    return Qobj([[cos(theta), -sin(theta)],[sin(theta),cos(theta)]]).tidyup()
```

```
Rp(pi/2)
```

```
V==Rp(pi/2)*H
```

```
# Solution Goes Here
H==Rp( -pi/2)*V
```

0.23.4 1) Using the $\hat{R}_p(\theta)$ operator, verify the operator properties described in Sections 4.1 and 4.2. Specifically, verify Eqns. 4.6, 4.7, 4.16, 4.18, 4.22, and 4.27

```
# Solution Goes Here
# 4.6
(Rp(pi/2) + Rp(pi/3)) * P45 == Rp(pi/2)*P45 + Rp(pi/3)*P45

#4.7
Rp(pi/3)*(P45 + H) == Rp(pi/3)*P45 + Rp(pi/3)*H

#4.16
H.dag()*Rp(pi/4).dag() == P45.dag()

#4.18
(Rp(pi/3)*Rp(pi/4)).dag() == (Rp(pi/3)*Rp(pi/4)).dag() == Rp(pi/4).dag()*Rp(pi/3).dag()

#4.22
Rp(pi/3).dag() * Rp(pi/3) == identity(2)

#4.27
Rp(pi/3).dag() == Rp( -pi/3)
```

0.23.5 Example: the similarity transform

The following defines a function that creates a similarity transform matrix. It takes the two old basis vectors and the two new basis vectors as arguments. To apply the transform, simply multiply the matrix onto the state vector or ooperator matrix. Following the examples below, explore this transform.

```
def sim_transform(o_basis1, o_basis2, n_basis1, n_basis2):
    a = n_basis1.dag()*o_basis1
    b = n_basis1.dag()*o_basis2
    c = n_basis2.dag()*o_basis1
    d = n_basis2.dag()*o_basis2
    return Qobj([[a,b],[c,d]])
```

We can define a similarity transform that converts from $HV \rightarrow \pm 45$

```
Shv45 = sim_transform(H,V,P45,M45)  # as found in Example 4.A.1, Eq. 4.A.10.
Shv45

Shv45 * H  # compare to Eq. 4.A.12
```

0.23.6 4) Use the similarity transform to represent $|V\rangle$ in the ± 45 basis

```
Shv45 * V
```

0.23.7 5) Represent $\hat{P}_H$ in the ± 45 basis.

Check your answer against Eqns. 4.A.17 and 4.72

```
Shv45 * Ph * Shv45.dag()
```

0.23.8 6) Represent $\hat{P}_V$ in the ± 45 basis.

```
Shv45 * Pv * Shv45.dag()
```

0.24 Lab 3: Operators

An overview of operator properties

```
import matplotlib.pyplot as plt
from numpy import sqrt,cos,sin,pi,arange
from qutip import *

H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
P45 = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
M45 = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
R = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
L = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])
```

0.24.1 Example 1: the outer product and the projection operator

We already have the $|H\rangle$ state represented as a vector in the HV basis, so the $\hat{P}_H$ operator is the outer product $|H\rangle\langle H|$:

```
Ph = H*H.dag()
Ph
```

Same with the $\hat{P}_V$ operator:

```
Pv = V*V.dag()
Pv
```

0.24.2 1) Verify Eq. 4.38 for the HV basis states. Repeat for the 45, and ± basis

0.24.3 2) Represent the $\hat{R}_p(\theta)$ operator in the HV basis and verify your representation by operating on $|H\rangle$ and $|V\rangle$ states.

```
def Rp(theta):
    return Qobj([[cos(theta), -sin(theta)],[sin(theta),cos(theta)]]).tidyup()

Rp(pi/2)

V == Rp(pi/2)*H
```

0.24.4 3) Using your $\hat{R}_p(\theta)$ operator, verify the operator properties described in Sections 4.1 and 4.2. Specifically, verify Eqns. 4.5, 4.6, 4.7, 4.16, 4.18, 4.22, and 4.27

0.24.5 Example: similarity transform

The following defines a function that creates a similarity transform matrix. It takes the two old basis vectors and the two new basis vectors.

```
def sim_transform(o_basis1, o_basis2, n_basis1, n_basis2):
    a = n_basis1.dag()*o_basis1
    b = n_basis1.dag()*o_basis2
    c = n_basis2.dag()*o_basis1
    d = n_basis2.dag()*o_basis2
    return Qobj([[a.data[0,0],b.data[0,0]],[c.data[0,0],d.data[0,0]]])
```

```
Shv45 = sim_transform(H,V,P45,M45)  # as found in Example 4.A.1, Eq. 4.A.10.
Shv45
```

```
Shv45 * L  # compare to Eq.
```

0.24.6 4) Represent $\hat{P}_H$ in the $\pm$45 basis.

Check your answer against Eqns. 4.A.17 and 4.72

```
Shv45*Ph*Shv45.dag()
```

0.25 Lab 4 - Measurements

First our standard definitions:

```
import matplotlib.pyplot as plt
from numpy import sqrt,pi,cos,sin,arange,random
from qutip import *
```

```
H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
P45 = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
M45 = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
R = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
L = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])
```

```
def sim_transform(o_basis1, o_basis2, n_basis1, n_basis2):
    a = n_basis1.dag()*o_basis1
    b = n_basis1.dag()*o_basis2
    c = n_basis2.dag()*o_basis1
    d = n_basis2.dag()*o_basis2
    return Qobj([[a.data[0,0],b.data[0,0]],[c.data[0,0],d.data[0,0]]])
```

0.25.1 Q: Define the $\hat{\mathcal{P}}_{HV}$ operator

```
Phv = H*H.dag() - V*V.dag()
Phv
```

0.25.2 Q: What is the expectation value $\langle\hat{\mathcal{P}}_{HV}\rangle$ for state $|\psi\rangle = \frac{1}{\sqrt{5}}|H\rangle + \frac{2}{\sqrt{5}}|V\rangle$? Interpret this result given the amplitudes in the state.

```
psi = 1/sqrt(5)*H + 2/sqrt(5)*V
```

```
psi.dag()*Phv*psi
```

0.25.3 Q: What is the variance of $\mathcal{P}_{HV}$?

```
psi.dag()*Phv*Phv*psi
```

```
1 - ( -0.6)**2
```

0.25.4 Example: Use the random function to generate a mock data set for the state $|\psi\rangle$.

```
random.choice([1, -1],size=10,p=[0.2,0.8])
```

gives a list of 10 numbers, either 1 or -1 with the associated probability p:

```
data = random.choice([1, -1],size=1000000,p=[0.2,0.8])
```

0.25.5 Q: Verify the mean and variance of the mock data set match your QM predictions. How big does the set need to be for you to get ±5% agreement?

```
data.mean()
```

```
data.var()
```

0.25.6 Q: Answer problems 5.11, 5.12, 5.13, 5.14 from the textbook. These are an opportunity to practice with a new operator $\hat{\mathcal{P}}_C$

0.26 Lab 4 - Measurements

First our standard definitions:

```
import matplotlib.pyplot as plt
from numpy import sqrt,pi,cos,sin,arange,random,exp
from qutip import *

H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
P45 = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
M45 = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
R = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
L = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])

def sim_transform(o_basis1, o_basis2, n_basis1, n_basis2):
    a = n_basis1.dag()*o_basis1
    b = n_basis1.dag()*o_basis2
    c = n_basis2.dag()*o_basis1
    d = n_basis2.dag()*o_basis2
    return Qobj([[a,b],[c,d]])

def Delta(state, op):
    """Calculate std. dev. of an observable in a given state"""
    eO2 = state.dag()*op*op*state
    eO = state.dag()*op*state
    return sqrt(eO2 - (eO)**2)
```

Q: Define the $\hat{\mathcal{P}}_{HV}$ operator

```
Phv = H*H.dag() - V*V.dag()
Phv
```

Q: What is the expectation value $\langle\hat{\mathcal{P}}_{HV}\rangle$ for state $|\psi\rangle = \frac{1}{\sqrt{5}}|H\rangle + \frac{2}{\sqrt{5}}|V\rangle$? Interpret this result given the amplitudes in the state.

```
psi = 1/sqrt(5)*H + 2/sqrt(5)*V

psi.dag()*Phv*psi
```

###Q: What is the variance of $\mathcal{P}_{HV}$?

```
psi.dag()*Phv*Phv*psi

1.0 - ( -0.6)**2
```

Ex: Use the random function to generate a mock data set for the state $|\psi\rangle$.

```
random.choice([1, -1],size=10,p=[0.2,0.8])
```

gives a list of 10 numbers, either 1 or -1 with the associated probability p:

```
data = random.choice([1, -1],size=20,p=[0.2,0.8])

data.mean()

data.var()
```

Q: Verify the mean and variance of the mock data set match your QM predictions. How big does the set need to be for you to get $\pm 5\%$ agreement?

```
data = random.choice([1, -1],size=10000,p=[0.2,0.8])

data.mean()

data.var()
```

10,000 does pretty well for getting to the predictions. "There is no substitute for an adequate sample size."

Q: Answer problems 5.11, 5.12, 5.13, 5.14, 5.17, 5.18, 5.19 from the textbook. These are an opportunity to practice with a new operator $\hat{\mathcal{P}}_C$

```
# 5.11
P_45 = P45*P45.dag() - M45*M45.dag()

# 5.12
P_c = L*L.dag() - R*R.dag()

# 5.13
(Phv*P_45 - P_45*Phv) == 2j * P_c

# 5.14
(Phv*P_c - P_c*Phv) == -2j * P_45

# 5.15
P_p45 = P45*P45.dag()
P_v = V*V.dag()

P_p45*P_v - P_v*P_p45
```

0.26.1 5.19 - this one is tricky, but is easier with our custom function

```
psi = 1/sqrt(3)*H + sqrt(2/3.0)*exp(1j*pi/3.0)*V
psi.norm()

Delta(psi,P_45)*Delta(psi,Phv)

1/2j*(psi.dag()*(Phv*P_45 - P_45*Phv)*psi)
```

0.27 Two-particle systems

An introduction to multi-particle spaces, starting with photon polarization states. This lab answers the question: How do we describe the state of two photons?

```
import matplotlib.pyplot as plt
from numpy import sqrt,pi,sin,cos,arange
from qutip import *
```

The polarization states (in the HV-basis):

```
H = basis(2,0)
V = basis(2,1)
P45 = 1/sqrt(2)*(H+V)
M45 = 1/sqrt(2)*(H -V)
L = 1/sqrt(2)*(H+1j*V)
R = 1/sqrt(2)*(H -1j*V)
```

Define two-particle states using the `tensor()` function:

Mathematically, we are taking the tensor product of two vectors. That product is a larger vector with twice as many entries as the individual state vectors. As long as we take the tensor products in the right order (i.e. always talking about photon 1 and photon 2 in that order) we can also make operators that act on two-photon states). In order to keep a consistent naming scheme, we'll call the first photon the **signal** photon and the second photon the **idler** photon. The names aren't particularly important but they come from the process we use in the lab: Spontaneous Parametric Down Conversion

First, look at a generic pair of vectors and their tensor product:

```
A = Qobj([[1],[2]])
B = Qobj([[3],[4]])
print(A)
print(B)
print(tensor(A,B))
```

So we see that the tensor product has the following elements: 1*3 = 3, 1*4 = 4, 2*3 = 6, 2*4 = 8. Essentially, we distributed the multiplication of the first vector through the second vector. Using the technical terms of vector spaces, the tensor product exists in a larger Hilbert space (the number of dimensions is the product of the dimensions of the original states). See this with larger initial states: two 3-dim vectors have a tensor product in 9-dim space:

```
C = Qobj([[1],[2],[3]])
D = Qobj([[4],[5],[6]])
print(tensor(C,D))
```

Now, back to the quantum mechanics. Form the four different combinations of two photons:

```
HH = tensor(H,H)
HV = tensor(H,V)
VH = tensor(V,H)
VV = tensor(V,V)

# How do we represent HH? It is a vector with four elements.
HH
```

So we interpret the state $|HH\rangle$ as the vector (1,0,0,0) in a four-dimensional space.

Recall: The polarization measurement operator (for one photon):

```
Phv = H*H.dag() - V*V.dag()
Phv
```

Also, the identity is defined as `qeye(n)` for `n` dimensions in qutip:

```
qeye(2)  # 2 -dimensional identity
```

The two-photon operator, measuring the **signal** photon, is formed with the `tensor()` function. It is the tensor product of the projection operator `Phv` and the 2-dimensional identity operator `qeye(2)`. The trick is putting them in the correct order. The first element in the tensor product acts on the signal photon, the second acts on the idler photon. So to act on only the signal photon, we create a tensor product with the projection operator first, and the identity second:

```
Phv_s = tensor(Phv,qeye(2))
Phv_s
```

It can be hard to interpret these values visually but remember it was constructed by multiplying all the terms between two matrices with only diagonal elements. It makes sense that the result is also diagonal. Also, the sign of the diagonal depends on the state of the signal photon (the first one listed). Recall the states are in the order: HH, HV, VH, VV so the first two states have H signal photons and are therefore 1, and the second two states are V signal photons so -1 for those diagonals.

Now construct the two-photon operator that measures the idler photon:

```
Phv_i = tensor(qeye(2),Phv)
Phv_i
```

Next, construct a projection operator that projects the idler photon to H:

```
Ph = H*H.dag()
Ph_i = tensor(qeye(2),Ph)  # Ph for idler photon
```

And the same but for the signal photon:

```
Ph_s = tensor(Ph,qeye(2))  # Ph for signal photon
```

You start to see the pattern. Build these up from our earlier operators, just apply them to the specific particle by including them in the tensor product at that position.

Next we will do some example calculations.

Example: find the probability of measuring a horizontal idler photon if the system is prepared in the state $|HH\rangle$

```
HH.dag()*Ph_i*HH
```

Example: find the probability of measuring a horizontal idler photon in the state $|\psi\rangle = |H, +45\rangle$

```
psi = tensor(H,P45)  # the prepared state

psi.dag()*Ph_i*psi
```

Example 8.2 prob. of measuring vertical signal and horizontal idler if $|\psi\rangle = |R, +45\rangle$

```
# First, form the prepared state:
psi = tensor(R,P45)

# Then create the projection operator for the state we are asking about:
projection = VH*VH.dag()

# Finally, calculate the probability by computing the bra -ket:
psi.dag()*projection*psi
```

0.27.1 Entangled states:

A very interesting system can be set up where there are paired photons being created with unknown but correlated polarization. In this case, we can say the state is in a combination of $|HH\rangle$ and $|VV\rangle$. If either two-photon state is allowed, then the normalized state is

$$|\phi^+\rangle = \frac{1}{\sqrt{2}}(|HH\rangle + |VV\rangle) \tag{9}$$

```
phiPlus = 1/sqrt(2)*(HH + VV)
```

```
phiPlus.dag()*Ph_i*phiPlus  # probability of measuring a horizontal idler photon:
```

This is expected, because the HH state has 50% of the probability amplitude. Same for a horizontal signal photon:

```
phiPlus.dag()*Ph_s*phiPlus  # probability of measuring a horizontal signal photon
```

0.27.2 Now, find $P(H_s|H_i)$ (Example 8.5)

```
# Projection operator for H idler and H signal:
phh = HH*HH.dag()
phiPlus.dag()*phh*phiPlus
```

```
# Projection operator for H idler
Pih = tensor(qeye(2),H*H.dag())
phiPlus.dag()*Pih*phiPlus
```

$P(H_s|H_i) = \frac{P(H_s,H_i)}{P(H_i)}$

```
0.5/0.5
```

Guaranteed to measure a horizontal signal photon whenever a horizontal idler photon is measured. What about vertical? Find the conditional probability of measuring a vertical signal photon if the idler photon is found to be vertical:

Now, measure a different basis (use the +45 states) to show that the photons are always found in the same polarization even when measured at a different angle:

```
# Solution

# Probability that signal is +45 and idler +45
Pp45p45 = tensor(P45,P45) * tensor(P45,P45).dag()
phiPlus.dag()*Pp45p45*phiPlus
```

```
# Solution

# Probability that the idler is +45 regardless of the signal
Pp45i = tensor(qeye(2),P45) * tensor(qeye(2),P45).dag()
phiPlus.dag()*Pp45i*phiPlus
```

Finally, to really drive this odd point home, show that they are **never** found in the $|+45,-45\rangle$ state:

```
# Solution

# Probability that they are in different 45 states:
Pp45m45 = tensor(P45,M45) * tensor(P45,M45).dag()

phiPlus.dag()*Pp45m45*phiPlus
```

0.27.3 Using these states solve problems 8.2, 8.3, 8.7, 8.8

0.28 Lab 6 - Spin

A few new operators (or new names for the same ones!) The three axes, x, y, z spin components can be measured with SA_x, SA_y, and SA_z devices.

We'll use $\hbar = 1$ for numerical results, this is fairly standard practice, but can be tricky to remember.

```
from numpy import sin,cos,pi,sqrt
from qutip import *

pz = Qobj([[1],[0]])  # +z
mz = Qobj([[0],[1]])  # -z
px = Qobj([[1/sqrt(2)],[1/sqrt(2)]])  # +x
mx = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])  # -x
py = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])  # +y
my = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])  # -y
Sx = 1/2.0*sigmax()
Sy = 1/2.0*sigmay()
Sz = 1/2.0*sigmaz()
```

0.28.1 Example: determine $P(S_x = \frac{\hbar}{2}|| - y\rangle)$

```
expect(px*px.dag(),my)
```

0.28.2 Example: verify the commutation relation: $\left[\hat{S}_x, \hat{S}_z\right] = -i\hbar\hat{S}_y$

```
Sx*Sz - Sz*Sx == -1j*Sy  # remember, h = 1
```

0.28.3 Ex: find $\langle\hat{S}_x\rangle$ for the state $|\psi\rangle = |+Z\rangle$.

```
pz.dag()*Sx*pz
```

```
expect(Sx,pz)
```

This makes sense given that S_x can be either $\frac{+\hbar}{2}$ or $\frac{-\hbar}{2}$ with equal probability. Similarly, if the state is $|\psi\rangle = |+x\rangle$.

```
px.dag()*Sx*px
```

```
expect(Sx,px)
```

Again, in units of $\hbar$.

0.28.4 Lab instructions:

1. Calculate the expectation value $\langle S_z\rangle$ for the states $|+x\rangle, |+y\rangle, |+z\rangle$
2. Calculate the expectation value $\langle S_z\rangle$ for the state $|\psi\rangle = \frac{1}{\sqrt{2}}|+x\rangle + \frac{1}{\sqrt{2}}|-x\rangle$. Interpret this result, does it make sense? Is there a simpler way to write this state?
3. Represent the spin-squared operator $\hat{S}^2$ as a matrix.
4. Verify that $\hat{S}^2$ commutes with each spin component: $\hat{S}_x, \hat{S}_y, \hat{S}_z$
5. Find the expectation value of S^2 for the three postive spins: $|+x\rangle, |+y\rangle, and|+z\rangle$. Interpret your results. Do the values differ? Should they?
6. Problem 6.9, 6.11, 6.12 from Chapter 6

0.29 Lab 7 - Time Evolution

Exploring time evolution of quantum states. Run the usual imports, and use the spin-1/2 states as previously defined:

```
import matplotlib.pyplot as plt
from numpy import sqrt,pi,arange,cos,sin,array
from qutip import *

%matplotlib inline

pz = Qobj([[1],[0]])
mz = Qobj([[0],[1]])
px = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
mx = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
py = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])
my = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
Sx = 1/2.0*sigmax()
Sy = 1/2.0*sigmay()
Sz = 1/2.0*sigmaz()
```

Define the Hamiltonian:

$$H = -\mu \cdot \mathbf{B} = -\gamma S_z B \tag{10}$$

$$\hat{H} = -\Omega \hat{S}_z \tag{11}$$

```
Omega = 5
H = -Omega*Sz

t = arange(0,4*pi/Omega,0.05)
```

The next line calls a Schrödinger equation solver (sesolve). It's arguments are the Hamiltonian, the starting state $|+x\rangle$ (px), the time values, and a list of operators. `sesolve` returns many things, but the `expect` method is most useful, it gives the expectation values of the three operators in the operator list.

```
expect_ops = [Sx,Sy,Sz]
result1 = sesolve(H, px, t, e_ops=expect_ops)

expect_ops[0] # TODO get name of variable to use in label

labels = ['x','y','z']
for r,l in zip(result1.expect,labels):
    result1_times = array(result1.times)
    plt.plot(result1_times*Omega/pi, r, label="$\\langle S_%c \\rangle $" % l)


plt.xlabel("Time ($\\Omega t/\\pi$)", size=18)
plt.legend(fontsize=16) #TODO fix legend text size
```

Now what if the system starts in $|+z\rangle$?

```
result2 = sesolve(H, pz, t, e_ops=[Sx,Sy,Sz])

for r,l in zip(result2.expect,labels):
    plt.plot(array(result2.times)*Omega/pi, r, label="$\\langle S_%c \\rangle $" % l)


plt.xlabel("Time ($\\Omega t/\\pi$)", size=18)
plt.legend(fontsize=16) #TODO fix legend text size
```

Spin-up stays spin-up (i.e. no prescession)

0.29.1 Two particle systems:

$|\psi\rangle = \frac{1}{\sqrt{2}}|+z,-z\rangle + \frac{1}{\sqrt{2}}|-z,+z\rangle$

Use the `tensor` QuTiP function to form multi-particle states

```
psi = 1/sqrt(2)*tensor(pz, mz) + 1/sqrt(2)*tensor(mz, pz)
```

Hamiltonian is the same for both particles so we use the tensor to form $\hat{H}$ from individual operators

```
omega = 5
H = -omega*tensor(Sz,Sz)

expect_op = tensor(mz,pz)*tensor(mz,pz).dag()
result3 = sesolve(H, psi, t, e_ops=expect_op)

for r,l in zip(result3.expect,labels):
    plt.plot(array(result3.times)*omega/pi, r, label="$\\langle -z,+z\\rangle$")

plt.xlabel("Time ($\\Omega t/\\pi$)", size=18)
plt.legend(fontsize=16) #TODO fix legend text size
```

The value is constant since the state is initially in an eigenstate of $\hat{H}$.

0.29.2 What if the magnetic field is not along an axis?

Notice the Hamiltonian has an x and a z component:

```
omega=2
H = -omega/sqrt(2)*(Sz + Sx)
t = arange(0,2*pi/omega,0.05)

result4 = sesolve(H, px, t, e_ops=[Sx, Sy, Sz])

for r,l in zip(result4.expect,labels):
    plt.plot(array(result4.times)*omega/pi, r, label="$\\langle S_%c \\rangle $" % l)

plt.xlabel("Time ($\\Omega t/\\pi$)", size=18)
plt.legend(fontsize=16) #TODO fix legend text size
```

Harder to interpret, so we'll use the Bloch sphere:

```
sx, sy, sz = result4.expect

b = Bloch()
b.add_points([sx,sy,sz])
b.zlabel = ['$\\left|+z\\right>$', '$\\left| -z\\right>$']
b.view = [ -45,20]
b.add_vectors([1/sqrt(2),0,1/sqrt(2)])
b.show()
```

0.29.3 Time-dependent Hamiltonian:

We'll explore the parameters of a spin in a time-varying magnetic field. This system is relevant to nuclear magnetic resonance (NMR) which is used in chemistry and as Magnetic Resonance Imaging (MRI) in medicine.

Following Compliment 9.A the Hamiltonian is:

$$\hat{H} = -\Omega_0 \hat{S}_z - \Omega_1 cos(\omega t)\hat{S}_x \tag{12}$$

We then solve for a certain amount of time after the state starts in $|\psi(0)\rangle = |+z\rangle$

We also use the definition of the Rabi frequency: $\Omega_R = \sqrt{(\omega - \Omega_0)^2 + (\Omega_1/2)^2}$ as in (9.A.28)

Note that the time span is 3 units of $\frac{2\pi}{\Omega_R}$. Leave the scaling in place, but to plot a longer time period, change 3.0 to something larger. This lets us match the units in Fig. 9.A.1.

```
omega0 = 2.0 * 2 * pi   # pick a nice value for a frequency, note this is 1 Hz
omega1 = 0.25 * 2 * pi  # 25% of omega0
w = 2.0 * 2 * pi        # the driving frequency

H0 = - omega0 * Sz      # the first term in H
H1 = - omega1 * Sx      # the second term in H

omegaR = sqrt((w - omega0)**2 + (omega1/2.0)**2)
t = arange(0,3.0 * 2 * pi / omegaR,0.02)   # scale the time by omegaR, plot 3 units of 2pi/ome

args = {'w': w}         # args dict

def H1_coeff(t, w):
    return cos(w * t)

H = [H0, [H1, H1_coeff]]
```

The next line calls a Schrödinger equation solver (`sesolve`). The arguments are the Hamiltonian, the starting state $|+z\rangle$ (`pz`), the time values, a list of operators, and the arguments to the function `H_t`. `sesolve` returns many things, but the `expect` method is most useful, it gives the expectation values of the four operators in the operator list. Notice the fourth operator is the $|-z\rangle$ projection operator. It's expectation value is $P(|-z\rangle, t)$

```
result5 = sesolve(H, pz, t, e_ops=[Sx, Sy, Sz, mz*mz.dag()], args=args)
sx, sy, sz, Pmz = result5.expect
```

Look at the Bloch sphere for this system:

```
b2 = Bloch()
b2.add_points([sx,sy,sz])
b2.zlabel = ['$\\left|+z\\right>$', '$\\left| -z\\right>$']
b2.show()
```

Make a plot analogous to Fig 9.A.1:

```
plt.tick_params(labelsize=18)
plt.plot(array(result5.times)*omegaR/pi,Pmz)
plt.xlabel("Time ($\\Omega_R t/\\pi$)", size=18)
plt.ylabel("$P( -z, t)$", size=18)
```

Q) What happens in each unit of time ($\frac{2\pi}{\Omega_R}$)? Look at the plot of $P(-z,t)$ to interpret this. How is your figure different from the on in Fig. 9.A.1?

Q) How does the evolution change if you double Ω_0?

Q) After doubling Ω_0 what if you double the driving frequency (ω) also? Interpret this observation in terms of Fig. 9.A.2. In practice, what experimental parameter changes Ω_0?

Q) How does Ω_1 influence the dynamics? (Be careful reading the plots since the units are scaled by Ω_R).

Advanced topic: we can change the Hamiltonian so the applied field turns off at a certain time, and it is possible to get the spin to stay in a particular state. This is very useful in quantum optics where certain operations change the atomic state in a very specific way.

```
omega0 = 1.0 * 2 * pi   # pick a nice value for a frequency, note this is 1 Hz
omega1 = 0.05 * 2 * pi  # 25% of omega0
w = 1.0 * 2 * pi        # the driving frequency

H0 = - omega0 * Sz      # the first term in H
H1 = - omega1 * Sx      # the second term in H

omegaR = sqrt((w - omega0)**2 + (omega1/2.0)**2)
t = arange(0,3.0 * 2 * pi / omegaR,0.05)   # scale the time by omegaR, plot 3 units of 2pi/ome

args = {
    'w': w,
    'omegaR': omegaR
}

def H1_coeff2(t, **kwargs):       # this function calculates H at each time step t
    if t < 2*pi/omegaR * 0.5:  # only add the H1 piece for the first chunk of time.
        coeff = cos(w * t)
    else:
        coeff = 0
    return coeff

H = [H0, [H1, H1_coeff2]]

result6 = sesolve(H, pz, t, e_ops=[Sx, Sy, Sz, mz*mz.dag()], args=args)
sx, sy, sz, Pz = result6.expect

plt.plot(result6.times,Pz)
plt.ylim( -0.1,1.1)
plt.xlim( -5,125)
plt.xlabel("Time ($\\Omega_R t/\\pi$)", size=18)
plt.ylabel("$P( -z, t)$", size=18)
```

0.30 Lab 8 - Simple Harmonic Oscillator states

Problems from Chapter 12

```
from numpy import sqrt
from qutip import *
```

0.30.1 Define the standard operators

```
N = 10  # pick a size for our state -space
a = destroy(N)
n = a.dag()*a
```

0.30.2 Problem 12.1:

```
a*a.dag() - a.dag()*a
```

0.30.3 Problem 12.2:

```
n*a.dag() - a.dag()*n
```

```
n*a.dag() - a.dag()*n == a.dag()
```

0.30.4 Problem 12.3 (use n=2 as a test-case):

To define $|2\rangle$ use the `basis(N,n)` command where `N` is the dimension of the vector, and `n` is the quantum number.

```
psi = basis(N,2)
```

0.30.5 Problem 12.5 and 12.6:

These are simple, just view the matrix representation of the operators

0.30.6 Problem 12.7:

First, define $\hat{X}$ and $\hat{P}$ operators

```
X=
P=
```

```
psi = 1/sqrt(2)*(basis(N,1)+basis(N,2))
```

0.30.7 Problem 12.8:

```
psi = 1/sqrt(2)*(basis(N,2)+basis(N,4))
```

0.31 Lab 8 - Simple Harmonic Oscillator states

Problems from Chapter 12

```
from numpy import sqrt
from qutip import *
```

0.31.1 Define the standard operators

```
N = 10  # pick a size for our state -space
a = destroy(N)
n = a.dag()*a
```

0.31.2 Problem 12.1:

```
a*a.dag() - a.dag()*a
```

0.31.3 Problem 12.2:

```
n*a.dag() - a.dag()*n
```

```
n*a.dag() - a.dag()*n == a.dag()
```

0.31.4 Problem 12.3 (use n=2 as a test-case):

To define $|2\rangle$ use the `basis(N,n)` command where `N` is the dimension of the vector, and `n` is the quantum number.

```
psi = basis(N,2)
```

```
psi
```

```
a.dag()*psi
```

```
a.dag()*basis(N,2) == sqrt(3)*basis(N,3)
```

0.31.5 Problem 12.5 and 12.6:

These are simple, just view the matrix representation of the operators

```
a
```

```
a.dag()
```

0.31.6 Problem 12.7:

First, define $\hat{X}$ and $\hat{P}$ operators

```
X = 1/2 * (a + a.dag())
P = 1/2j * (a - a.dag())
```

```
psi = 1/sqrt(2)*(basis(N,1)+basis(N,2))
ex = psi.dag()*X*psi  # or expect(X, psi)
exq = psi.dag()*X*X*psi  # expect(X*X, psi)
ep = psi.dag()*P*psi  # expect(P, psi)
epq = psi.dag()*P*P*psi  # or expect(P*P, psi)
```

```
deltaX = sqrt(exq - ex**2)
deltaP = sqrt(epq - ep**2)

deltaX * deltaP * 2 # compare to uncertainty relation ( \Deltax \Deltap >= 1/2)
# the factor of two is to convert from the unitless version of the operator
```

Alternatively, we can find the indeterminacy bound for ΔX and ΔP (the unitless operators):

$$\Delta X \Delta P \geq \frac{1}{2}\left|\left\langle\left[\hat{X},\hat{P}\right]\right\rangle\right| \tag{13}$$

```
1/2*(psi.dag()*commutator(X,P)*psi)
```

Which is also satisfied by the calculated value (1.41 >0.25)

0.31.7 Problem 12.8:

```
psi = 1/sqrt(2)*(basis(N,2)+basis(N,4))
ex = expect(X, psi)
exq = expect(X*X, psi)
ep = expect(P, psi)
epq = expect(P*P, psi)

deltaX = sqrt(exq - ex**2)
deltaP = sqrt(epq - ep**2)

deltaX * deltaP * 2 # to compare to book solution which uses the full x and p operators with u
```

0.32 Schrodinger Equation

```
%pylab inline
from qutip import *

pz = Qobj([[1],[0]])
mz = Qobj([[0],[1]])
px = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
mx = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
py = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])
my = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
Sx = 1/2.0*sigmax()
Sy = 1/2.0*sigmay()
Sz = 1/2.0*sigmaz()
```

Schrödinger solver

```
H = -0.2*sigmaz()

e_list = 0.5*sigmax()  # list of expectation values to calculate

t_list = linspace(0,100,1000)  # list of times to evaluate

psi = px  # initial state

result = sesolve(H, psi, t_list, e_list)

plot(result.expect[0])
```

$< \hat{S}_x >$ average oscillates - this is QM "precession"

```
result2 = sesolve(H, pz, t_list, e_list)

plot(result2.expect[0])
```

Sx averages to zero at all times for a +Z state in a Bz field.

0.33 Coupled Systems

```
%pylab inline
from qutip import *

wc = 1 * 2*pi
wa = 1 * 2*pi
gamma = 0.05
g = gamma * 2*pi
N = 15
kappa = 0.005
n_th_a = 1.0


a = tensor(destroy(N),qeye(2))
sm = tensor(qeye(N),destroy(2))
sz = tensor(qeye(N),sigmaz())

H = wc*a.dag()*a + wa*sm.dag()*sm + g*(a.dag()*sm + a*sm.dag())

tlist = linspace(0,50,200)

c_ops = []

# cavity relaxation
rate = kappa * (1 + n_th_a)
if rate > 0.0:
    c_ops.append(sqrt(rate) * a)

# cavity excitation, if temperature > 0
rate = kappa * n_th_a
if rate > 0.0:
    c_ops.append(sqrt(rate) * a.dag())

# qubit relaxation
rate = gamma
if rate > 0.0:
    c_ops.append(sqrt(rate) * sm)

psi0 = tensor(basis(N,0),basis(2,1))  # start with an excited atom
output = mesolve(H, psi0, tlist, c_ops, [a.dag() * a, sm.dag() * sm])

n_c = output.expect[0]
n_a = output.expect[1]

fig, axes = subplots(1, 1, figsize=(10,6))

axes.plot(tlist, n_c, label="Cavity")
axes.plot(tlist, n_a, label="Atom excited state")
axes.legend(loc=0)
axes.set_xlabel('Time')
axes.set_ylabel('Occupation probability')
axes.set_title('Vacuum Rabi oscillations')
```

0.34 Jones Calculus (Vectors and Matrices)

Useful for working examples and problems with polarization

```
import matplotlib.pyplot as plt
from numpy import *
from qutip import *
```

These are the polarization states:

```
H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
P45 = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
M45 = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
R = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
L = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])
```

```
R
```

Devices:
HWP - Half-wave plate axis at θ to the horizontal
LP - Linear polarizer, axis at θ
QWP - Quarter-wave plate, axis at θ
Note, these are functions so you need to call them with a specific value of theta.

```
def HWP(theta):
    return Qobj([[cos(2*theta),sin(2*theta)],[sin(2*theta), -cos(2*theta)]]).tidyup()
```

```
HWP(0)
```

```
def LP(theta):
    return Qobj([[cos(theta)**2,cos(theta)*sin(theta)],[sin(theta)*cos(theta),sin(theta)**2]])
```

```
def QWP(theta):
    return Qobj([[cos(theta)**2 + 1j*sin(theta)**2,
                  (1 -1j)*sin(theta)*cos(theta)],
                 [(1 -1j)*sin(theta)*cos(theta),
                  sin(theta)**2 + 1j*cos(theta)**2]]).tidyup()
```

Try a linear polarizer at $+45°$ acting on a vertical beam

```
out = LP(pi/4)*V
out
```

Have to interpret this result: it is a constant times the D polarization state:

```
1/sqrt(2)*P45
```

Notice, that it is also a combination of the H and V states:

```
1/2.0*H + 1/2.0*V
```

This should be somewhat obvious from the vector itself.
Next, find out the intensity of the light that is transmitted. This is given by the "norm" of the vector, sometimes also called the magnitude: $|\mathbf{a}|$

```
out.norm()  # equivalent to sqrt(0.5**2 + 0.5**2)
```

0.35 Measurement simulation

A way to simulate data from measurements of a specific quantum state.

Start with standard imports:

```
import matplotlib.pyplot as plt
from numpy import sqrt,pi,cos,sin,arange,random,real,imag
from qutip import *
%matplotlib inline
```

Define several standard states, these are photon polarization states:

```
H = Qobj([[1],[0]])
V = Qobj([[0],[1]])
P45 = Qobj([[1/sqrt(2)],[1/sqrt(2)]])
M45 = Qobj([[1/sqrt(2)],[ -1/sqrt(2)]])
R = Qobj([[1/sqrt(2)],[ -1j/sqrt(2)]])
L = Qobj([[1/sqrt(2)],[1j/sqrt(2)]])

# Define the Phv measurement operator:
Phv = H*H.dag() - V*V.dag()
Phv
```

Define a quantum state:

$$|\psi\rangle = \frac{1}{\sqrt{5}}|H\rangle + \frac{2}{\sqrt{5}}|V\rangle \tag{14}$$

```
psi = 1/sqrt(5)*H + 2/sqrt(5)*V
psi

# The function to generate a mock data set:
def simulateData(state,oper,size=10000):
    """Generate a simulated data set given a state and measurement operator.

    state -> the prepared state
    oper -> the measurement operator

    Example:
    H = Qobj([[1],[0]])
    V = Qobj([[0],[1]])
    psi = 1/sqrt(5)*H + 2/sqrt(5)*V
    Phv = H*H.dag() - V*V.dag()
    data = simulateData(psi,Phv)

    will generate 10000 values in the data array that obey the probability defined in the stat
    """
    A = basis(2,0)
    B = basis(2,1)
    allowed_results = [A.dag()*oper*A, B.dag()*oper*B]
    probability_amps = [A.dag()*state, B.dag()*state]
    pvals = [abs(pa.conjugate()*pa) for pa in probability_amps]
    data = random.choice(allowed_results,size=size,p=pvals)
    return data

data = simulateData(psi,Phv)
```

```
print("Variance: ",data.var())
print("Mean: ",data.mean())

plt.hist(real(data))
```

0.36 Gaussian Wavepackets

```
import numpy as np

def Gaussian(x, chi, sigma):
    return 1/(sigma*sqrt(2*np.pi)) * np.exp( -(x - chi)**2/(2*sigma**2))

%pylab inline

x = arange(0,10,0.1)

plot(x,Gaussian(x,5,0.5))

import scipy.integrate as integ

integ.quad?

integ.quad(Gaussian, -numpy.inf, numpy.inf, (5,5))

def eipx_Gaussian(x,p,chi,sigma):
    return np.exp(1j*p*x)*Gaussian(x,chi,sigma)

def psip(p):
    result = scipy.integrate.quad(eipx_Gaussian, -numpy.inf,numpy.inf,(p,5,0.5))
    return result[0]

plot?



psip(0)

from sympy import *

z = symbols('z')

integrate(Gaussian(z,5,0.5),(z, -oo,oo))
```

0.37 Gaussian Wavepackets (using Sympy)

```
from sympy import *

%matplotlib inline

def Gaussian(x, alpha, beta):
    return 1/(beta*sqrt(pi)) * exp( -(x - alpha)**2/(beta**2))

x,p,a,b = symbols('x p a b')

plot(Gaussian(x,5,0.5))

integrate(Gaussian(x,5,0.5),(x, -oo,oo))

def eipx_Gaussian(x,p,alpha,beta):
    return exp(1j*p*x)*Gaussian(x,alpha,beta)

def psip(p):
    result = integrate(eipx_Gaussian,(p, -oo,oo))
    return result[0]

eipx_Gaussian(2,1,2,3)

integrate(eipx_Gaussian(x,p,a,b),(p, -oo,oo),conds='none')
```